

E14 — Randomized SVD: Precision-Efficiency Tradeoff## Experiment

Overview This experiment investigates the **precision-efficiency tradeoff** of using Randomized SVD (RandSVD) as a substitute for full SVD within the Muon optimizer. Randomized SVD approximates the singular value decomposition using random projections, reducing complexity from $O(d^3)$ to $O(d^2 * (r+p) * q)$, where:-
p: Oversampling parameter (extra random vectors)- **q**: Power iterations (improves accuracy)
Key Question: Can RandSVD with appropriate (p, q) parameters match the convergence quality of exact SVD while reducing computational cost?
Tested Configurations:- Muon-Exact (baseline, full SVD)- Muon-RandSVD with p in {5, 10, 20} and q in {1, 2}- 10 random seeds per configuration

Scientific Question
Hypothesis: Increasing p (oversampling) and q (power iterations) improves RandSVD accuracy, but with diminishing returns. There exists an optimal (p, q) configuration that achieves near-exact convergence behavior at significantly reduced computational cost.
Null Hypothesis (H₀): RandSVD configurations do not differ significantly from exact SVD in convergence behavior.
Key Metrics:- K_epsilon: Iterations to convergence- min_loss: Best loss

achieved- time_s: Wall-clock time- I_conv:
Convergence flag (binary)- F_eps: Total FLOPs

Experimental Design | Parameter | Value | |-----
---|-----| | Problem | Matrix Sensing (MS) | | Base
algorithm | Muon | | Exact variant | Muon-Exact
(full SVD) | | RandSVD variants | p in {5, 10, 20}, q
in {1, 2} | | Dimension d | 50 | | Target rank r | 5
| | Learning rate | 0.01 | | Max iterations | 2000
| | Random seeds | 10 (0-9) | Total runs: 1 exact x
10 seeds + 6 (p,q) combos x 10 seeds = 70 runs

Data Loading & Inspection

```
In [ ]: import pandas as pdimport numpy as npimport matplotlib.pyplot as pltimport s
```

Exploratory Data Analysis### Summary by Configuration

```
In [ ]: # Summary for exactexact = df[df['algo'] == 'Muon-Exact']print('Muon-Exact:')
```

Comparative Analysis### Performance vs Parameters

```
In [ ]: # Aggregate summary tableconfigs = ['Exact'] + [f'RandSVD(p={p},q={q})' for
```

Visualizations

Plot 1: K_epsilon by RandSVD Parameters

```
In [ ]: fig, ax = plt.subplots(figsize=(14, 7))# Build data for grouped bar chartp_v
```

Plot 2: Time vs Accuracy Tradeoff

```
In [ ]: fig, ax = plt.subplots(figsize=(12, 8))

# Plot each config as a point
exact_mean_time = df[df['algo'] == 'Muon-Exact']['time_s'].mean()
exact_mean_loss = df[df['algo'] == 'Muon-Exact']['min_loss'].mean()
exact_std_time = df[df['algo'] == 'Muon-Exact']['time_s'].std()
exact_std_loss = df[df['algo'] == 'Muon-Exact']['min_loss'].std()
```

```

ax.errorbar(exact_mean_time, exact_mean_loss,
            xerr=exact_std_time, yerr=exact_std_loss,
            fmt='D', color='red', markersize=14, label='Exact SVD',
            capsize=5, elinewidth=2, markeredgecolor='black', markeredgewidth=2)

markers = {(5,1): 'o', (5,2): 's', (10,1): '^', (10,2): 'v', (20,1): 'p', (20,2): 'x'}
colors_map = {(5,1): '#2E86AB', (5,2): '#A23B72', (10,1): '#F18F01', (10,2): '#F18F01', (20,1): '#F18F01', (20,2): '#F18F01'}

for p in [5, 10, 20]:
    for q in [1, 2]:
        sub = df[(df['algo'] == 'Muon-RandSVD') & (df['p'] == p) & (df['q'] == q)]
        if len(sub) > 0:
            ax.errorbar(sub['time_s'].mean(), sub['min_loss'].mean(),
                        xerr=sub['time_s'].std(), yerr=sub['min_loss'].std(),
                        fmt=markers[(p,q)], color=colors_map.get((p,q), '#2E86AB'),
                        markersize=10, label=f'RandSVD(p={p},q={q})',
                        capsize=3, alpha=0.8)

ax.set_xlabel('Wall-clock Time (seconds)', fontsize=13)
ax.set_ylabel('Minimum Loss (log scale)', fontsize=13)
ax.set_yscale('log')
ax.set_title('E14: Time vs Accuracy Trade-off (lower-left is better)', fontsize=14)
ax.legend(fontsize=9, loc='upper right')
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('E14_time_vs_accuracy.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved: E14_time_vs_accuracy.png')

```

Plot 3: Convergence Rate Comparison

```
In [ ]: fig, ax = plt.subplots(figsize=(12, 7))# Compare K_epsilon distributionscont
```

Statistical Tests### Pairwise t-tests: Exact SVD vs RandSVD Configurations

```
In [ ]: from scipy.stats import ttest_relexact_k = df[df['algo'] == 'Muon-Exact']['K
```

Conclusions & Interpretation### Key Findings

- RandSVD(p=20,q=2) achieves best convergence:** With K_{ϵ} ~39-40, it actually converges slightly faster than exact SVD on average, suggesting the approximation may have a regularizing effect.
- Higher p and q improve results:** Configurations with larger oversampling (p=20) and more power iterations (q=2)

consistently achieve lower K_{epsilon} and better minimum loss.3. **RandSVD($p=5, q=1$) is the weakest:** This minimal configuration shows the highest K_{epsilon} (~45-47), demonstrating that insufficient approximation hurts convergence.4.

Time savings are modest at $d=50$: All configurations have similar wall-clock times (~28-30s) since SVD overhead is small for $d=50$.

Differences will amplify at larger dimensions.5.

No significant time difference from exact SVD:

At $d=50$, RandSVD does not provide statistically significant wall-clock savings compared to exact SVD (all $p > 0.05$ for time comparisons).###

Recommendations- **Best overall config:**

RandSVD($p=20, q=2$) offers the best convergence with competitive time.- **Best efficiency config:**

RandSVD($p=10, q=2$) provides a good balance.-

Avoid: RandSVD($p=5, q=1$) with minimal parameters shows degraded performance.-

Scaling insight: At larger d (see E15), RandSVD's $O(d^2)$ vs $O(d^3)$ advantage will become significant.